

4

DECODING AMERICAN CIVIL WAR CIPHERS



Cryptography is the science of secure communication through the use of *codes* and *ciphers*. A code replaces whole words with other words; a cipher scrambles or replaces the letters in words (so technically, Morse code is really Morse cipher). One goal of cryptography is to use a *key* to both *encrypt* readable *plaintext* into unreadable *ciphertext* and then *decrypt* it back to plaintext. The goal of *cryptanalysis* is to decode ciphers and codes without knowing their key or encryption algorithm.

In this chapter, we'll investigate two ciphers used in the American Civil War: the route cipher, used by the North, and the rail fence cipher, used by both sides. We'll also look at what made one so successful and how we can use lessons learned from its application to better write programs for inexperienced users and those unfamiliar with your Python code.

Project #8: The Route Cipher

In the American Civil War, the Union had just about every advantage over the Confederacy, including the field of cryptography. The Union had better codes, better ciphers, and better-trained personnel. But perhaps its biggest advantage was in leadership and organization.

The head of the US Military Telegraph Department was Anson Stager (**Figure 4-1**). As the cofounder of Western Union, Stager knew from experience that telegraph operators made fewer mistakes when sending

whole words, as opposed to the strings of random letters and numbers common to most ciphertext. He also knew that military dispatches only needed to stay secret long enough for orders to be carried out. His secure solution was a hybrid cryptosystem called the *route transposition cipher*, a combination of transposed real words and code words that became one of the most successful military ciphers of all time.



Figure 4-1: General Anson Stager, US Telegraph Corps, 1865

Transposition ciphers *scramble* the arrangement of letters or words, unlike substitution ciphers, which *replace* the letters in the plaintext with different characters or symbols. **Figure 4-2** shows an example of a route transposition cipher. The message is written left to right over a number of predetermined columns and rows, important plaintext words are replaced by code words, and the last row is filled with dummy placeholder words. The reader determines the order of the rearranged words by

traversing up and down these columns, as shown. The starting word is *REST*, and then the encryption route is shown with arrows.

Code Words

VILLAGE = Enemy	ROANOKE = Cavalry
GODWIN = Tennessee	SNOW = Rebels

Original Message in Encryption Matrix

Enemy	cavalry	heading	to
Tennessee	With	Rebels	gone
you	are	free	to
transport	your	supplies	south

Encryption Route + Code & Dummy Words

VILLAGE	ROANOKE	heading	to
GODWIN	With	SNOW	gone
you	are	free	to
transport	your	supplies	south
REST	IS	JUST	FILLER

Cyphertext

REST	TRANSPORT	YOU	GODWIN	VILLAGE
ROANOKE	WITH	ARE	YOUR	IS
JUST	SUPPLIES	FREE		
SNOW	HEADING	TO	GONE	TO
SOUTH	FILLER			

Figure 4-2: A route cipher using actual Union code words

To fully decode this message, you need to know both the starting point and route used to traverse the message and create the final cyphertext *and* the meaning of the code words.

In the early 20th century, the distinguished military cryptanalyst William Friedman disparaged Stager’s route cipher. He considered it too unsophisticated and found it highly improbable that the Confederates never cracked it. But the fact remains that hundreds of thousands of route ciphers sent during the war were apparently never decoded, and not from lack of trying. In an early example of crowdsourcing, the Confederates

published the coded messages in newspapers, hoping for some help with the decryption, but to no avail. While some historians speculate that this cipher was broken at times, Stager's design teaches several important lessons:

Design for human error. Military ciphers have to be simple, as hundreds might be sent in a day. The real words used in the route cipher made it much less likely to be garbled by telegraph operators. Stager knew his customer and designed for them. He recognized the limitations of his workforce and tailored his product accordingly. The Confederates, by contrast, had great difficulty deciphering their own complex messages, sometimes giving up and riding around enemy lines to talk face-to-face!

Innovation trumps invention. Sometimes you don't need to invent something new; you just need to rediscover something old. The short word-transposition ciphers suitable for telegraph transmission were too weak to use on their own, but combined with code names and disruptive dummy words, they confounded the Confederates.

Share learning. Because everyone in the Telegraph Corps used the same methodology, it was easy to build on existing techniques and share lessons learned. This allowed the route cipher to evolve over time with the introduction of slang and intentional misspellings, as well as a growing number of code words for places, people, and dates.

Stager's practical cipher may not have pleased later "purists," but it was the perfect design for the time. The concepts behind it are timeless and easily transferable to modern-day applications.

THE OBJECTIVE

In Harry Turtledove's award-winning 1992 novel *Guns of the South*, time travelers provide Confederate armies with modern weaponry, changing the course of history. Instead of AK-47s, let's pretend you've traveled back to 1864 with your laptop, a few extra batteries, and Python to design an algorithm that will decrypt a route cipher based on an assumed encryption matrix and path. In the spirit of Stager, you'll write a user-friendly program that will reduce human error.

The Strategy

When it comes to solving ciphers, it's a lot easier if you know what type you're dealing with. In this case, you know it's a transposition cipher, because it's composed of real words that are jumbled. You also know there are code words and null words present. Your job is to figure out ways to decrypt the *transposition* part of the route cipher and let someone else worry about code words while you go have a well-deserved mint julep.

Creating a Control Message

To understand how to do this, create your own message and route cipher. Call this your *control message*:

- Number of columns = 4
- Number of rows = 5
- Start position = Bottom left
- Route = Alternating up and down columns
- Plaintext = 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
- Ciphertext = 16 12 8 4 0 1 5 9 13 17 18 14 10 6 2 3 7 11 15 19
- Key = -1 2 -3 4

Using a numeric progression for the plaintext allows you to instantly tell whether you've gotten all or part of the decryption correct, at any place within the message.

The transposition matrix is shown in **Figure 4-3**. The gray arrows indicate the encryption route.

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15
16	17	18	19

= 16 12 8 4 0 1 5 9 13 17 18 14 10 6 2 3 7 11 15 19

Figure 4-3: The transposition matrix for the control message with the route cipher path and resulting ciphertext

The key keeps track of both the *order* and *direction* of the route through the columns. The route doesn't have to move through the columns in order. For instance, it can move down the first column, up the third, down the fourth, and finally up the second. Negative numbers mean you start at the bottom and read up a column; positive numbers mean the reverse. For the control message, the final key used in the program will be a list: [-1, 2, -3, 4]. This list will instruct the program to start reading up from the bottom of column 1, move to the top of column 2 and read down, move to the bottom of column 3 and read up, and move to the top of column 4 and read down.

Note that you shouldn't use 0 in keys because the users, being human, prefer to start counting at 1. Of course, Python prefers to start counting at 0, so you'll need to subtract 1 from the key values behind the scenes. Everybody wins!

Later, in “**Route Transposition Cipher: Brute-Force Attack**” on [page 88](#), you can use this compact key structure to brute-force your way through a route cipher, automatically trying hundreds of keys until the plaintext is restored.

Designing, Populating, and Depopulating the Matrix

You'll input the ciphertext as a continuous string. For your program to unravel the route through this string, you'll first need to build and populate a translation matrix. The ciphertext string is just the columns in the transposition matrix in [Figure 4-3](#) laid end to end, in the order they were read. And as there are five rows in the transposition matrix, every group of five elements in the ciphertext represents a separate column. You can represent this matrix with a list of lists:

```
>>> list_of_lists = [['16', '12', '8', '4', '0'], ['1', '5', '9', '13', '17'],  
                    ['18', '14', '10', '6', '2'], ['3', '7', '11', '15', '19']]
```

The items in this new list now represent lists—with each list representing a column—and the five elements in each list represent the rows that comprise that column. This is a little hard to see, so let's print each of these nested lists on a separate line:

```
>>> for nested_list in list_of_lists:
    print(nested_list)
[16, 12, 8, 4, 0]
[1, 5, 9, 13, 17]
[18, 14, 10, 6, 2]
[3, 7, 11, 15, 19]
```

If you read each list left to right, starting at the top, you follow the transposition route, which was up and down alternate columns (see **Figure 4-3**). From Python's point of view, the first column read is `list-of-lists[0]`, and the starting point is `list-of-lists[0][0]`.

Now, normalize the route by reading all columns in the same direction as the starting column (up). This requires reversing the order of elements in every other list, as shown in bold here:

```
[16, 12, 8, 4, 0]
[17, 13, 9, 5, 1]
[18, 14, 10, 6, 2]
[19, 15, 11, 7, 3]
```

A pattern emerges. If you start at the upper right and read down each column, ending at the lower left, the numbers are in numerical order; you've restored the plaintext!

To replicate this, your script can loop through every nested list, removing the last item in that list and adding the item to a new string, until the translation matrix has been emptied. The script will know from the key which nested lists it needs to reverse and the order in which to depopulate the matrix. The output will be a string of the restored plaintext:

```
'0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19'
```

You should now have a very general view of the strategy. Let's get more descriptive and write the pseudocode next.

The Pseudocode

The script can be broken up into three major parts: user input, translation matrix population, and decryption to plaintext. You should be able to see these parts in the following pseudocode:

```
Load the ciphertext string.
Convert ciphertext into a cipherlist to split out individual words.
Get input for the number of columns and rows.
Get input for the key.
Convert key into a list to split out individual numbers.
Create a new list for the translation matrix.
For every number in the key:
    Create a new list and append every  $n$  items ( $n$  = # of rows) from the cipherlist
    Use the sign of key number to decide whether to read the row forward or backward
    Using the chosen direction, add the new list to the matrix. The index of each
    new list is based on the column number used in the key.
Create a new string to hold translation results.
For range of rows:
    For the nested list in translation matrix:
        Remove the last word in nested list
        Add the word to the translation string.
Print the translation string.
```

Everything before the first loop is essentially just collecting and reformatting the cipher data. The first loop is responsible for building and populating the matrix, and the second loop creates a translation string from that matrix. Finally, the translation string is printed.

The Route Cipher Decryption Code

Listing 4-1 takes a message encrypted with the route cipher, the number of columns and rows in the transposition matrix, and a key and then displays the translated plaintext. It will decrypt all “common” route ciphers, where the route starts at the top or bottom of a column and continues up and/or down columns.

This is the prototype version; once you’re sure it’s working, you’ll package it for others to use. You can download this code at

<https://www.nostarch.com/impracticalpython/>.


```
❶ ciphertext = "16 12 8 4 0 1 5 9 13 17 18 14 10 6 2 3 7 11 15 19"

    # split elements into words, not letters

❷ cipherlist = list(ciphertext.split())

❸ # initialize variables
    COLS = 4
    ROWS = 5
    key = '-1 2 -3 4' # neg number means read UP column vs. DOWN
    translation_matrix = [None] * COLS
    plaintext = ''
    start = 0
    stop = ROWS

    # turn key_int into list of integers:
❹ key_int = [int(i) for i in key.split()]

    # turn columns into items in list of lists:
❺ for k in key_int:
    ❻ if k < 0: # reading bottom-to-top of column
        col_items = cipherlist[start:stop]
    elif k > 0: # reading top-to-bottom of column
        col_items = list((reversed(cipherlist[start:stop])))
    translation_matrix[abs(k) - 1] = col_items
    start += ROWS
    stop += ROWS

    print("\nciphertext = {}".format(ciphertext))
    print("\ntranslation matrix =", *translation_matrix, sep="\n")
    print("\nkey length = {}".format(len(key_int)))

    # loop through nested lists popping off last item to new list:
❺ for i in range(ROWS):
    for col_items in translation_matrix:
        ❽ word = str(col_items.pop())
        ❾ plaintext += word + ' '
```

```
print("\nplaintext = {}".format(plaintext))
```

Listing 4-1: Code for route_cipher_decrypt_prototype.py

Start by loading the ciphertext ❶ as a string. You want to deal with words, not letters, so split the string apart based on empty spaces using the `split()` string method to create a new list named `cipherlist` ❷. The `split()` method is the inverse of the `join()` method, which you’ve seen before. You can split on any string; the method just defaults to runs of consecutive whitespace, deleting each whitespace before it moves to the next.

Now it’s time to input what you know about the cipher ❸: the columns and rows, which form the matrix, and the key, which contains the route. Initialize the column and row numbers as constants. Then make an empty list named `translation_matrix` to hold the contents of each column as a (nested) list. Assign placeholders by multiplying the value `None` by the number of columns. You can use the indexes of these empty items to put columns back in their correct order for keys that are not in numerical order.

An empty string named `plaintext` will hold the decrypted message. Next are some slicing parameters. Note that some of these are derived from the number of rows, which equates to the number of items in each column.

Now, convert the key variable, which is a string, into a list of integers using *list comprehension*—a shorthand way of performing operations on lists ❹. You’ll use the numbers in the key as indexes later, so they need to be integers.

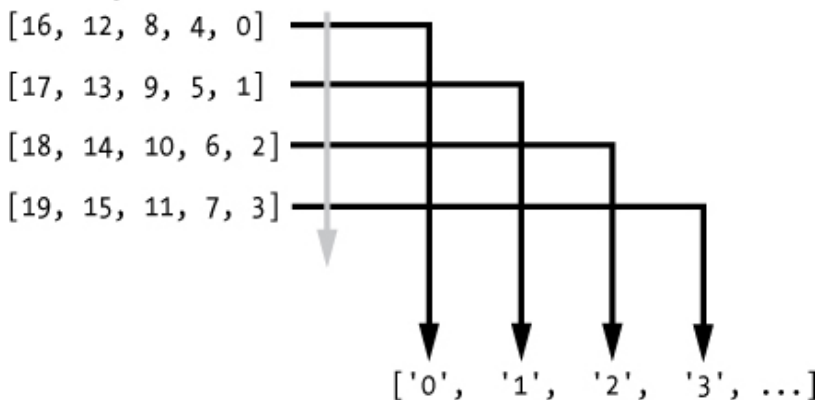
The next block of code is a `for` loop that populates the `translation_matrix`, which is just a list of lists ❺. Since each column becomes a nested list and the length of the `key_int` list is equal to the number of columns, the range for the loop is the key, which also describes the route.

Inside the loop, use a conditional to check whether the key is positive or negative ❻; if the key is positive, then the direction of the slice is reversed. Assign the slice to the correct position in `translation_matrix` based on the absolute key value and subtract 1 (since the keys don’t include 0, but

the list indexes do). Finish the loop by advancing the slice endpoints by the number of rows and printing some useful information.

The final block ⑦ loops through the number of rows—which is equivalent to the number of words in one of the nested lists—and through each nested list. The first two of these loops are shown in **Figure 4-4**. As you stop in each nested list, you get to employ one of my favorite Python functions, the list `pop()` method ⑧. The `pop()` method removes and returns the last item from a list, unless a specific index is provided. It destroys the nested list, but you're done with it anyway.

First loop



Second loop

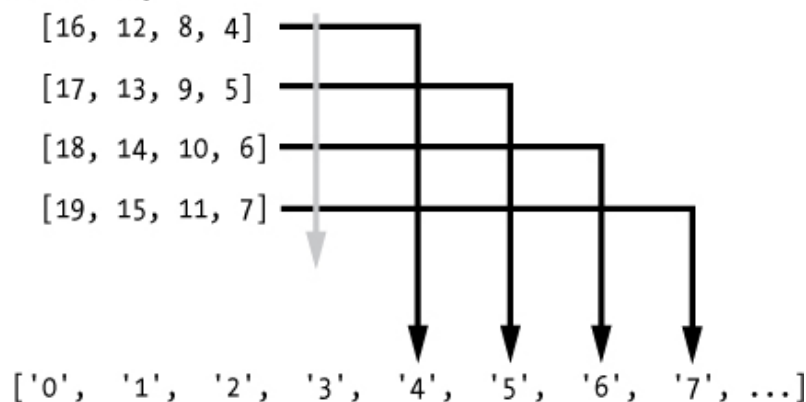


Figure 4-4: First and second loop through the nested lists, removing and appending each end item to the translation string

As soon as you pop off a word, concatenate it to the `plaintext` string and add a space ⑨. All that's left to do is display the decrypted ciphertext. The output for the numeric test set looks like this:

```
plaintext = 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
```

That looks like success!

Hacking the Route Cipher

The preceding code assumes you know the route through the encryption matrix or have correctly guessed the key. If those assumptions aren't true, your only recourse is to try every possible key and matrix arrangement. You'll get a chance to automate the key selection process—for a given number of columns—in “**Route Transposition Cipher: Brute-Force Attack**” on **page 88**. But, as you'll see, the Union route cipher is well fortified against brute-force attacks. You can crack it, but you'll end up with so much data, you'll feel like the dog that chased a car and caught it.

As messages get longer, the number of possible encryption paths in a transposition cipher becomes too large for a brute-force solution, even using modern computers. For example, if there are eight columns, and you allow the route to skip to any column, the number of ways to combine the columns is the factorial of eight: $8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 40,320$. That's 40,320 paths *before* you start choosing alternative routes through the columns. If the route can change direction up or down a column, the number of combinations increases to 10,321,920. And if you consider starting *anywhere* in a column—instead of at the very top or bottom—and allow *any* route through the matrix (such as spiraling), things will really start to get out of hand!

For this reason, even short transposition ciphers can have thousands to millions of possible paths. Even if the number of paths is manageable for a computer and a brute-force attack may prevail, you'll still need a way to sift through the myriad outcomes and either choose a winner computationally or select a small subset of candidates to visually examine.

For the more common *letter*-transposition ciphers, it's easy to write a function that detects English by comparing each decryption attempt to a dictionary file. If the number of decrypted words belonging to a dictionary is greater than a certain threshold percentage, you've probably cracked the cipher. Likewise, if there's a high frequency of common letter pairs (*digrams*)—like *er*, *th*, *on*, or *an*—you may have found the solution. Unfortunately, this approach won't work for a *word*-transposition cipher like the one you're using here.

Dictionaries can't help you figure out whether words have been arranged correctly. For word arrangement, you could try using approaches like

grammatical rules and probabilistic language models, such as *n*-grams, combing through thousands of decrypts and picking candidate outcomes programmatically, but Stager's wise use of code names and dummy words in his route cipher will greatly complicate the process.

Cryptanalysts consider short, straight-up transposition ciphers to be fairly easy to crack without a computer, despite the issues just described. They look for common word or letter pairs that make sense, and use those to guess the number of rows in the transposition matrix.

To illustrate, let's use our control message composed of numbers. In

Figure 4-5, you can see the ciphertext outcomes for a 4x5 matrix, each produced by starting at one of the four corners of the grid, and following an alternating, sequential route. All cases include repetition of adjacent numbers (shaded in **Figure 4-5**). These indicate where you're moving laterally through the grid, and they provide clues to the design of the matrix and the route taken through it. You can immediately see that there were five rows, because the first of each common pair is a fifth word.

Moreover, knowing there are 20 words in the message, you learn the number of columns was four ($20 / 5 = 4$). Using the reasonable assumption that the plaintext message was written left to right, you can even guess the route. For example, if you start in the lower right, you go up to 3, then left to 2, then down to 18, then left to 17, then up to 1 and left to 0. Of course, this would be more difficult with words, since the connection between words isn't as explicit, but using numbers really makes the point.

5 digits = 5 rows!

Lower right = 19 15 11 7 3 2 6 10 14 18 17 13 9 5 1 0 4 8 12 16

Lower left = 16 12 8 4 0 1 5 9 13 17 18 14 10 6 2 3 7 11 15 19

Upper right = 3 7 11 15 19 18 14 10 6 2 1 5 9 13 17 16 12 8 4 0

Upper left = 0 4 8 12 16 17 13 9 5 1 2 6 10 14 18 19 15 11 7 3

Figure 4-5: Characters or words in logical order (shaded) can be used to guess an encryption route.

Look at **Figure 4-6**, based on the message in **Figure 4-2**. End words and possible linked words, like "is just" or "heading to," are shaded.

REST TRANSPORT YOU GODWIN VILLAGE ROANOKE WITH ARE YOUR
IS JUST SUPPLIES FREE SNOW HEADING TO GONE TO SOUTH FILLER

Figure 4-6: Human hacking of the route cipher in **Figure 4-2**. A five-row matrix is indicated.

There are 20 total words, for which there could be 4, 5, or 10 rows. It's doubtful that a two-column matrix would be used, so we are realistically dealing with a 4×5 or a 5×4 arrangement. If the route cipher path is like that in **Figure 4-5**, then we'd expect to see two nonshaded words between shaded words for a four-row matrix and three nonshaded words for a five-row matrix. It's harder to come up with sensible word pairs that honor the four-column pattern, regardless of which direction you read the ciphertext. So, we're probably dealing with a five-column solution that starts on the left side of the matrix—since the linked words make sense read left to right.

Note how the shaded words in **Figure 4-6** fill out the top and bottom rows in the transposition matrix in **Figure 4-7**. This is what we would expect, as the path is “turning around” at the top and bottom of every column. Graphical solutions: God's gift to the innumerate!

VILLAGE	ROANOKE	heading	to
GODWIN	With	SNOW	gone
you	are	free	to
transport	your	supplies	south
REST	IS	JUST	FILLER

Figure 4-7: Shaded words in **Figure 4-6** placed in the transposition matrix

This seemed easy, but then again, we know how a route cipher works. Confederate code breakers eventually discovered it too, but the use of code words denied them full entry into the system. To hack the codes, they needed a captured codebook or a large organization capable of acquiring and analyzing big data, which was beyond the reach of the Confederacy in the 19th century.

Adding a User Interface

The second goal of this project is to write the code in such a way as to reduce human error, especially from those with less experience (including technicians, interns, peers, and telegraph clerks in 1864). Of course, the

best way to make a program user-friendly is to include a *graphical user interface (GUI)*, but at times, this isn't practical or possible. For example, code-cracking programs automatically loop through thousands of possible keys, and autogenerating these keys is easier than getting them directly from a user.

In this example, you'll proceed with the assumption that the user will crack open the program file and enter some input or even make minor code changes. Here are some guidelines to follow:

1. Start with a useful docstring (see **Chapter 1**).
2. Place all required user input at the top.
3. Use comments to clarify input requirements.
4. Clearly separate user input from the remaining code.
5. Encapsulate most processes in functions.
6. Include functions to catch predictable user errors.

The nice thing about this approach is that no one's intelligence gets insulted. If a user *wants* to scroll down and look at the code, or even change it, there's nothing stopping them. If all they want to do is enter some values and get a black box solution, then they're happy, too. And we've honored the spirit of Anson Stager by making things simple and reducing the chance for error.

Instructing the User and Getting Input

Listing 4-2 shows the prototype code repackaged for sharing with others.

You can find this code at

<https://www.nostarch.com/impracticalpython/>.

route_cipher_decrypt.py, part 1

- ```
❶ """Decrypt a path through a Union Route Cipher.
```
- ```
❷ Designed for whole-word transposition ciphers with variable rows & columns.
```
- ```
Assumes encryption began at either top or bottom of a column.
```
- ```
Key indicates the order to read columns and the direction to traverse.
```
- ```
Negative column numbers mean start at bottom and read up.
```
- ```
Positive column numbers mean start at top & read down.
```

Example below is for 4x4 matrix with key -1 2 -3 4.

Note "0" is not allowed.

Arrows show encryption route; for negative key values read UP.

```

1   2   3   4

  _ _ _ _
| ^ | | | ^ | | | MESSAGE IS WRITTEN
|_|_|_v_|_|_|_v_|
| ^ | | | ^ | | | ACROSS EACH ROW
|_|_|_v_|_|_|_v_|
| ^ | | | ^ | | | IN THIS MANNER
|_|_|_v_|_|_|_v_|
| ^ | | | ^ | | | LAST ROW IS FILLED WITH DUMMY WORDS
|_|_|_v_|_|_|_v_|
START           END
```

Required inputs - a text message, # of columns, # of rows, key string

Prints translated plaintext

```
"""
```

```
❸ import sys
```

```
#=====
```

```
❹ # USER INPUT:
```

```
❺ # the string to be decrypted (type or paste between triple-quotes):
```

```
ciphertext = """16 12 8 4 0 1 5 9 13 17 18 14 10 6 2 3 7 11 15 19
"""
```

```
❻ # number of columns in the transposition matrix:
```

```
COLS = 4
```

```
# number of rows in the transposition matrix:
```

```
ROWS = 5
```

```
❼ # key with spaces between numbers; negative to read UP column (ex = -1 2 -3 4):
```

```
key = """ -1 2 -3 4 """
```



```
❸ # END OF USER INPUT - DO NOT EDIT BELOW THIS LINE!
```

```
#=====
```

```
❹
```

Listing 4-2: Docstring, imports, and user input for route_cipher_decrypt.py

Start with a multiline docstring in triple quotes ❶. The docstring informs the user that the program only decrypts a typical route cipher—one that starts at either the top or bottom of a column—and how to enter the key information ❷. A diagram is included to help make the point.

Next, import `sys` for access to system fonts and functions ❸. You're going to check the user's input for acceptance criteria, so you need to display messages in the shell in eye-catching red. Putting this `import` statement here is a catch-22. Since the strategic goal is to hide the working code from the user, you really should apply this later in the program. But the Python convention of putting all `import` statements at the top is too strong to ignore.

Now for the input section. How often have you seen or dealt with code where inputs or changes have to be made *throughout* the program? This can be confusing for the author and even worse for another user. So, move all these important variables to the top for convenience, common courtesy, and error prevention.

First, separate the input section with a line and then let the user know that they're on deck with an all-caps comment ❹. The required inputs are clearly defined with comments. You can use triple quotes for the text input to better accommodate long snippets of text. Note that I've entered the string of numbers from **Figure 4-3** ❺. Next, the user needs to add the number of columns and rows for the transposition matrix ❻, followed by the proposed (or known) key ❼.

End the user input section with a declaration comment to that effect and a caution to not edit anything below the following line ❸. Then add some extra spaces to more clearly separate the input section from the rest of the program ❹.

Defining the main() Function

Listing 4-3 defines the `main()` function, which runs the program and prints out the plaintext after the cipher is decoded. The `main()` function can be defined before or after the functions it calls, as long as it is the last function called.

route_cipher_decrypt.py, part 2

```
def main():
    """Run program and print decrypted plaintext."""
    ❶ print("\nCiphertext = {}".format(ciphertext))
    print("Trying {} columns".format(COLS))
    print("Trying {} rows".format(ROWS))
    print("Trying key = {}".format(key))

    # split elements into words, not letters
    ❷ cipherlist = list(ciphertext.split())
    ❸ validate_col_row(cipherlist)
    ❹ key_int = key_to_int(key)
    ❺ translation_matrix = build_matrix(key_int, cipherlist)
    ❻ plaintext = decrypt(translation_matrix)

    ❼ print("Plaintext = {}".format(plaintext))
```

Listing 4-3: Defines the `main()` function

Start the `main()` function by printing the user input to the shell ❶. Then turn the ciphertext into a list by splitting on whitespace, as you did in the prototype code ❷.

The next series of statements call functions you will define shortly. The first checks whether the input rows and columns are valid for the message length ❸. The second converts the `key` variable from a string to a list of integers ❹. The third builds the translation matrix ❺, and the fourth runs the decryption algorithm on the matrix and returns a plaintext string ❻. Finish `main()` by printing the plaintext ❼.

Verifying Data

As you continue to package *route_cipher_decrypt.py* for the end user, you need to verify that the input is valid. **Listing 4-4** anticipates common user errors and provides the user with helpful feedback and guidance.

route_cipher_decrypt.py, part 3

```
❶ def validate_col_row(cipherlist):
    """Check that input columns & rows are valid vs. message length."""
    factors = []
    len_cipher = len(cipherlist)
    ❷ for i in range(2, len_cipher): # range excludes 1-column ciphers
        if len_cipher % i == 0:
            factors.append(i)
    ❸ print("\nLength of cipher = {}".format(len_cipher))
    print("Acceptable column/row values include: {}".format(factors))
    print()
    ❹ if ROWS * COLS != len_cipher:
        print("\nError - Input columns & rows not factors of length "
              "of cipher. Terminating program.", file=sys.stderr)
        sys.exit(1)

    ❺ def key_to_int(key):
        """Turn key into list of integers & check validity."""
        ❻ key_int = [int(i) for i in key.split()]
        key_int_lo = min(key_int)
        key_int_hi = max(key_int)
        ❼ if len(key_int) != COLS or key_int_lo < -COLS or key_int_hi > COLS \
            or 0 in key_int:
            ❽ print("\nError - Problem with key. Terminating.", file=sys.stderr)
            sys.exit(1)
        else:
            ❾ return key_int
```

Listing 4-4: Defines functions for checking and prepping user input

The `validate_col_row()` function checks that the input column and row numbers are appropriate for the length of the `cipherlist`, which you pass as an argument ❶. The transposition matrix is always the same size as the number of words in the message, so the number of columns and the number

of rows have to be a factor of the message size. To determine all the permissible factors, first make an empty list to hold the factors and then get the length of the cipherlist. Use the *cipherlist*, rather than the input *ciphertext*, as the elements in the ciphertext are *letters*, not words.

Normally, to get the factors of a number, you would use a range of (1, *number* + 1), but you don't want these endpoints in the *factors* list, because a translation matrix with these dimensions would just be the plaintext. So exclude these values from the range ❷. Since a factor of a number divides evenly into that number, use the modulo operator (%) to find the factors and then append them to the *factors* list.

Next, display some useful information for the user: the length of the cipherlist and the acceptable choices for rows and columns ❸. Finally, multiply the user's two choices together and compare the product to the length of the cipherlist. If they don't match, print a red warning message in the shell (using our old `file=sys.stderr` trick) and terminate the program ❹. Use `sys.exit(1)`, as the 1 indicates an abnormal exit.

Now define a function to check the key and convert it from a string to a list ❺. Pass it the *key* variable as an argument. Split out each item in *key* and convert it to an integer; name the list *key_int* to distinguish it from the user-entered *key* variable ❻. Next, determine the minimum and maximum values in the *key_int* list. Then use an *if* statement to make sure the list contains the same number of items as there are columns and none of the items in *key* is too large, too small, or equal to 0 ❼. Terminate the program with an error message if any of those criteria fail to pass ❽. Otherwise, return the *key_int* list ❾.

Building and Decoding the Translation Matrix

Listing 4-5 defines two functions, one to build the translation matrix and one to decode it, and calls the `main()` function as a module or in standalone mode.

route_cipher_decrypt.py, part 4

```
❶ def build_matrix(key_int, cipherlist):  
    """Turn every n items in a list into a new item in a list of lists."""
```

```

translation_matrix = [None] * COLS
start = 0
stop = ROWS
for k in key_int:
    if k < 0: # read bottom-to-top of column
        col_items = cipherlist[start:stop]
    elif k > 0: # read top-to-bottom of columnn
        col_items = list((reversed(cipherlist[start:stop])))
    translation_matrix[abs(k) - 1] = col_items
    start += ROWS
    stop += ROWS
return translation_matrix

❷ def decrypt(translation_matrix):
    """Loop through nested lists popping off last item to a string."""
    plaintext = ''
    for i in range(ROWS):
        for matrix_col in translation_matrix:
            word = str(matrix_col.pop())
            plaintext += word + ' '
    return plaintext

❸ if __name__ == '__main__':
    main()

```

Listing 4-5: Defines the functions for building and decoding the translation matrix

These two functions represent encapsulation of code in the `route_cipher_decrypt_prototype.py` program. See **Listing 4-1** for a detailed description.

First, define a function to build the translation matrix; pass it the `key_int` and `cipherlist` variables as arguments ❶. Have the function return the list of lists.

Next, bundle the decryption code, where you pop off the end of each nested list, as a function that uses the `translation_matrix` list as an argument ❷. Return the plaintext, which will be printed by the `main()` function.

End with the conditional statement that lets the program run as a module or in stand-alone mode ❸.

If you're an occasional or one-time user of this code, you'll appreciate how straightforward and approachable it is. If you plan to alter the code for your own purposes, you'll also appreciate that the key variables are accessible and the major tasks are modularized. You won't have to dig through the program to tease out what matters or understand the difference between arcane variables like `list1` and `list2`.

Here's the output of the program, using the ciphertext from **Figure 4-3**:

```
Ciphertext = 16 12 8 4 0 1 5 9 13 17 18 14 10 6 2 3 7 11 15 19

Trying 4 columns
Trying 5 rows
Trying key = -1 2 -3 4

Length of cipher = 20
Acceptable column/row values include: [2, 4, 5, 10]

Plaintext = 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
```

You should now be able to easily decrypt a route transposition cipher with a known key or test suspected routes by using the script's clear and accessible interface to adjust the key. You'll get a chance to truly crack one of these ciphers, by automatically trying every possible key, in **"Route Transposition Cipher: Brute-Force Attack"** on **page 88**.

Project #9: The Rail Fence Cipher

Confederate officers and spies were pretty much on their own when it came to cryptography. This led to unsophisticated solutions, one of the favorites being the rail fence cipher, so named due to its resemblance to the zigzag pattern of a split-rail fence (shown in **Figure 4-8**).



Figure 4-8: A rail fence

The rail fence is a simple-to-use transposition cipher, like the Union's route cipher, but differs from the route cipher in that it transposes letters rather than words, making it more error-prone. And since the number of possible keys is much more restrictive than the number of paths through a route cipher, the rail fence cipher is much easier to "tear down."

Both the Union and Confederates used the rail fence as a field cipher, and the spies probably didn't use code words very often. Codebooks needed to be tightly controlled, for obvious reasons, and were more likely to be secured in a military telegraph office than carried around by easily compromised undercover agents.

Sometimes the Confederates used the more complex Vigenère cipher (see **"Project #12: Hiding a Vigenère Cipher"** on **page 106**) for important messages—and some unimportant ones to mislead enemies—but it was tedious work to decipher and equally laborious to encrypt and not suitable for fast field communications.

Despite their lack of training in the mechanics of cryptography, the Confederacy, and Southerners in general, were clever and innovative. Among their more impressive accomplishments in the art of secret messages was the use of microphotography, 100 years before it was widely adopted during the Cold War.

THE OBJECTIVE

Write Python programs that will help a spy encrypt and decrypt a “two-rail” (two-row) rail fence cipher. You should write the programs in a way that will reduce potential errors by inexperienced users.

The Strategy

To encrypt a message with the rail fence cipher, follow the steps in **Figure 4-9**.

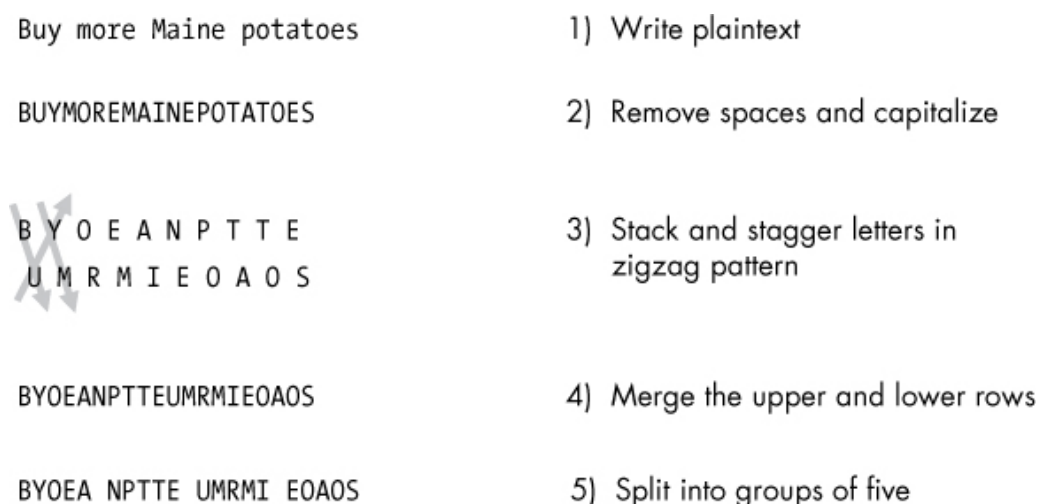


Figure 4-9: Encryption process for a “two-rail” rail fence cipher

After the plaintext is written, the spaces are removed, and all the letters are converted to uppercase (Step 2). Using uppercase is common convention in cryptography, as it obfuscates the presence of proper names and the beginning of sentences, giving a cryptanalyst fewer clues for deciphering the message.

The message is then written in stacked fashion, with every other letter below the previous letter and shifted over one space (Step 3). This is where the “rail fence” analogy becomes apparent.

The first row is then written, followed immediately by the second row on the same line (Step 4), and then the letters are broken into groups of five to create the illusion of distinct words and to further confuse the cryptanalyst (Step 5).

To decrypt a rail fence cipher, reverse the process. Just remove the spaces, divide the message in half, stack the second half below the first, offset by one letter, and read the message using the zigzag pattern. If the ciphertext has an odd number of letters, put the extra letter in the first (upper) half.

To make things easy for people who want to use a rail fence cipher, follow the preceding steps to write two programs, one to encrypt and another to decrypt. **Figure 4-9** is essentially your pseudocode, so let's get to it. And since you now know how to package the code for inexperienced users, take that approach from the start.

The Rail Fence Cipher Encryption Code

The code in this section allows a user to enter a plaintext message and have the encrypted results print in the interpreter window. This code is available for download with the book's resources at

<https://www.nostarch.com/impracticalpython/>.

Instructing the User and Getting Input

Listing 4-6, at the top of *rail_fence_cipher_encrypt.py*, provides the program's instructions and assigns the plaintext to a variable.

rail_fence_cipher_encrypt.py, part 1

```
❶ r"""Encrypt a Civil War 'rail fence' type cipher.

    This is for a "2-rail" fence cipher for short messages.

    Example text to encrypt:  'Buy more Maine potatoes'

    Rail fence style:  B Y O E A N P T T E
                      U M R M I E O A O S

    Read zigzag:       \/\\/\\/\\/\\/\//

    Encrypted:  BYOEANPTTE UMRMI EOSOS

    """
```

```

#-----
❷ # USER INPUT:

    # the string to be encrypted (paste between quotes):
❸ plaintext = """Let us cross over the river and rest under the shade of the tree
    """

❹ # END OF USER INPUT - DO NOT EDIT BELOW THIS LINE!
#-----

```

*Listing 4-6: Docstring and user input section for
rail_fence_cipher_encrypt.py*

Start with a multiline docstring, placing an `r` (which stands for “raw”) prefix before the first set of triple quotes ❶. Without this prefix, Pylint will complain bitterly about the `\\` slashes used further down. Fortunately, pydocstyle points this out so you can fix it (read **Chapter 1** to learn all about Pylint and pydocstyle). If you want to know more about *raw strings*, see Section 2.4.1 in the Python docs

(https://docs.python.org/3.6/reference/lexical_analysis.html#string-and-bytes-literals).

Next, separate the program’s docstring and `import` statements from the input section with a line and let the user know they’re on deck with an all-caps comment ❷. Clearly define the input requirement with comments and place the plaintext inside triple quotes to better accommodate long text strings ❸.

Finally, end the user input section with a declaration to that effect and caution to not edit anything below the following line ❹.

Encrypting a Message

Add **Listing 4-7** to *rail_fence_cipher_encrypt.py* in order to handle the encryption processes.

rail_fence_cipher_encrypt.py, part 2

```

❶ def main():
    """Run program to encrypt message using 2-rail rail fence cipher."""
    message = prep_plaintext(plaintext)
    rails = build_rails(message)
    encrypt(rails)

❷ def prep_plaintext(plaintext):
    """Remove spaces & leading/trailing whitespace."""
    ❸ message = "".join(plaintext.split())
    ❹ message = message.upper() # convention for ciphertext is uppercase
    print("\nplaintext = {}".format(plaintext))
    return message

❺ def build_rails(message):
    """Build strings with every other letter in a message."""
    evens = message[::2]
    odds = message[1::2]
    ❻ rails = evens + odds
    return rails

❽ def encrypt(rails):
    """Split letters in ciphertext into chunks of 5 & join to make string."""
    ❿ ciphertext = ' '.join([rails[i:i+5] for i in range(0, len(rails), 5)])
    print("ciphertext = {}".format(ciphertext))

❻ if __name__ == '__main__':
    main()

```

Listing 4-7: Defines functions to encrypt the plaintext message

To start, define a `main()` function to run the program ❶. Having a `main()` function gives you the flexibility to use this program as a module in another program later, should the need arise. This function calls three other functions: one to prepare the input plaintext, one to build the “rails” used by the cipher, and one to break the encrypted text into five-letter chunks.

Next, define a function to take the input string and prepare it for encryption ❷. This involves removing spaces ❸ and converting letters to upper-

case (as in Step 2 in **Figure 4-9**) ④. Then, after a newline, print the plaintext to the screen and return it.

Now, define a function to build two strings, as in Step 3 of **Figure 4-9**, by slicing `message` for evens (starting at 0 and stepping by 2) and for odds (starting at 1 and stepping by 2) ⑤. The two strings are then concatenated in a new string, named `rails` ⑥, and returned.

Define an encryption function that takes the `rails` string as an argument ⑦. Use list comprehension to split the ciphertext into chunks of five (as in Step 5 of **Figure 4-9**) ⑧. The encrypted text is then printed to the screen. Finish with the code to run the program as a module or in stand-alone mode ⑨.

Here is the output from this program:

```
plaintext = Let us cross over the river and rest under the shade of the trees
ciphertext = LTSRS OETEI EADET NETEH DOTER EEUCO SVRHR VRNRS UDRHS AEFHT ES
```

The Rail Fence Cipher Encryption Code

The code in this section allows a user to enter a message encrypted with the rail fence cipher and have the plaintext printed in the interpreter window. This code is available for download along with the rest of the book's resources at <https://www.nostarch.com/impracticalpython/>.

Importing Modules, Instructing the User, and Getting Input

Listing 4-8 starts with instructions similar to those in the `rail_fence_cipher_encrypt.py` program (**Listing 4-6**), imports two modules, and gets the user input.

rail_fence_cipher_decrypt.py, part 1

```
r"""Decrypt a Civil War 'rail fence' type cipher.

This is for a 2-rail fence cipher for short messages.

Example plaintext: 'Buy more Maine potatoes'
```

```

Rail fence style:  B Y O E A N P T T E
                  U M R M I E O A O S

Read zigzag:      \/\/\/\/\/\/\/\/\/

Ciphertext:  BYOEANPTTE UMRMI EOSOS

"""
❶ import math
   import itertools

   #-----
   # USER INPUT:

   # the string to be decrypted (paste between quotes):
❷ ciphertext = """"LTSRS OETEI EADET NETEH DOTER EEUCO SVRHR VRNRS UDRHS AEFHT ES
   """"

   # END OF USER INPUT - DO NOT EDIT BELOW THIS LINE!
   #-----

```

Listing 4-8: Imports modules, instructs the user, and gets user input

One difference here is that you need to import the `math` and `itertools` modules ❶. You'll use `math` for rounding. The `itertools` module is a group of functions in the Python Standard Library that create iterators for efficient looping. You'll use `itertools`'s `zip_longest()` function during the decryption process.

The only other change is that the user should enter ciphertext, rather than plaintext ❷.

Decrypting a Message

Listing 4-9 defines the functions for preparing and decoding the ciphertext and finishes off `rail_fence_cipher_decrypt.py`.

rail_fence_cipher_decrypt.py, part 2

```

❶ def main():
    """Run program to decrypt 2-rail rail fence cipher."""
    message = prep_ciphertext(ciphertext)
    row1, row2 = split_rails(message)
    decrypt(row1, row2)

❷ def prep_ciphertext(ciphertext):
    """Remove whitespace."""
    message = "".join(ciphertext.split())
    print("\nciphertext = {}".format(ciphertext))
    return message

❸ def split_rails(message):
    """Split message in two, always rounding UP for 1st row."""
    ❹ row_1_len = math.ceil(len(message)/2)
    ❺ row1 = (message[:row_1_len]).lower()
    row2 = (message[row_1_len:]).lower()
    return row1, row2

❻ def decrypt(row1, row2):
    """Build list with every other letter in 2 strings & print."""
    ❼ plaintext = []
    ❽ for r1, r2 in itertools.zip_longest(row1, row2):
        plaintext.append(r1)
        plaintext.append(r2)
    ❾ if None in plaintext:
        plaintext.pop()
        print("rail 1 = {}".format(row1))
        print("rail 2 = {}".format(row2))
        print("\nplaintext = {}".format(''.join(plaintext)))

❿ if __name__ == '__main__':
    main()

```

Listing 4-9: Prepares, decodes, and prints the message

The `main()` function here ❶ is similar to the one used in the encryption program in **Listing 4-7**. Three functions are called: one to prepare the input

string, one to “split the rails” in the rail fence cipher, and one to stitch the two rails back together into readable plaintext.

Start with a function that repeats the preprocessing steps used during encryption ❷. Remove the spaces between the five-letter chunks, as well as any other whitespace created during the pasting of the ciphertext, and print and return the ciphertext.

Next, you need to split the message back into two halves to reverse the encryption process ❸. As I mentioned in “**The Strategy**” on **page 81**, the extra letter in a message with an odd number of characters is assigned to the top row. To address the odd-numbered case, use the `math.ceil()` method ❹. “Ceil” stands for “ceiling,” so when you divide by 2, the answer is always rounded up to the nearest integer. Assign this number to the `row_1_len` variable. Once you know the length of the first row, use that value and slicing to divide the `message` variable into two strings representing the rows ❺. End the function by returning the row variables.

Now it’s just a matter of choosing and joining every other letter from the rows to stitch the plaintext back together. Define a `decrypt()` function and pass it the strings for `row1` and `row2` ❻. Start the translation process by making an empty list to hold the results ❼. Next, you need an easy way to deal with cases where the ciphertext has an odd number of letters—resulting in two rows of different lengths—because Python prevents you from looping through two uneven sequences by raising an index-out-of-range error. This is why we’ve imported the `itertools` module—its functions help with looping to circumvent this problem.

The `itertools.zip_longest()` function accepts two strings as arguments and loops through them without complaint, appending a null value (`None`) to the `plaintext` list when it gets to the end of the shorter string ❽. You don’t want to print this null value, so if it’s there, remove it using the `pop()` method you applied in the route cipher code ❾. Complete the decryption process by printing the two rows (rails) to the screen, followed by the decrypted ciphertext.

End with the standard code for running the program as a module or in stand-alone mode ❿. The output from the program is as follows:

```
ciphertext = LTSRS OETEI EADET NETEH DOTER EEUCO SVRHR VRNRS UDRHS AEFHT ES

rail 1 = LTSRSOETEIEADETNETEHDOTERE
rail 2 = EUCOSVRHRVRNRSUDRHSAEFHTES

plaintext = letuscrossovertheriverandrestundertheshadeofthetrees
```

Note that there will be no spaces between words, but that's okay—you don't want to leave the cryptanalyst feeling completely useless!

Summary

That completes our foray into Civil War ciphers. You wrote a program that helps a user decrypt a route cipher, and you gained valuable insights into how it works and how to hack it. You can implement an automated attack on the cipher in the following practice projects, but remember, with its many possible paths and use of code words, the Union route cipher is a tough nut to fully crack.

Next, you wrote programs to encrypt and decrypt two-rail fence ciphers. Given how tedious and error-prone the manual encryption and decryption processes are, having an automated way to do most of the work would have been valuable to both sides in the war. And to further address these types of issues, you wrote your code to be user-friendly to the inexperienced cryptanalyst or spy.

Further Reading

More beginner-level Python programs for working with transposition ciphers are available in *Cracking Codes with Python* (No Starch Press, 2018) by Al Sweigart.

Excellent and well-illustrated overviews of cryptography can be found in *Mysterious Messages: A History of Codes and Ciphers* (The Penguin Group, 2009) by Gary Blackwood and *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography* (Anchor, 2000) by Simon Singh.

The sites <http://www.civilwarsignals.org/pages/crypto/crypto.html> and http://www.mathaware.org/mam/06/Sauerberg_route-essay.html in-

clude descriptions of Edward Porter Alexander's attempt to solve a route cipher. Alexander was the father of the Confederate Army Signal Corps and a brilliant military innovator with many impressive accomplishments.

Practice Projects

Hone your cryptography skills with these projects. Solutions are available in the appendix and online.

Hacking Lincoln

In his book, *Mysterious Messages: A History of Codes and Ciphers*, Gary Blackwood reproduces an actual message sent by Abraham Lincoln and encrypted with a route cipher:

**THIS OFF DETAINED ASCERTAIN WAYLAND CORRESPONDENTS OF
AT WHY AND IF FILLS IT YOU GET THEY NEPTUNE THE TRIBUNE
PLEASE ARE THEM CAN UP**

Use the *route_cipher_decrypt.py* program to solve this cryptogram. The number of columns and rows must be factors of the message length, and the route starts in one of the corners, doesn't skip columns, and alternates direction with every column change. The code word definitions and plaintext solution can be found in the appendix.

Identifying Cipher Types

The sooner you know what type of cipher you're dealing with, the sooner you can break it. Word-transposition ciphers are easy to spot, but letter-transposition ciphers can look like letter-*substitution* ciphers. Fortunately, you can distinguish between the two by using the frequency of occurrence of letters in the ciphertext. Since the letters are just scrambled and not replaced in transposition ciphers, their frequency distribution will be the same as for the language in which the plaintext was written. An exception, however, is military messages, which use jargon and omit many common words. For these, you need a frequency table built from other military messages.

Write a Python program that takes a string of ciphertext as input and determines whether it is more likely to be a transposition cipher or a substi-

tution cipher. Test it with the files *cipher_a.txt* and *cipher_b.txt*, downloadable from <https://www.nostarch.com/impracticalpython/>. A solution can be found in the appendix and online at the book's website in *identify_cipher_type_practice.py*.

Storing a Key as a Dictionary

Write a short script to break a route cipher key into two parts: one to record the column order and one to record the direction to read through the rows in the column (up or down). Store the column number as a dictionary key and the reading direction as the dictionary value. Have the program interactively request the key value for each column from the user. A solution can be found in the appendix and online in the file *key_dictionary_practice.py*.

Automating Possible Keys

To attempt to decipher a route cipher using any combination of columns in its path, you need to know what those combinations are, so you can enter them as arguments in a decryption function. Write a Python program that accepts an integer (such as the number of columns) and returns a collection of tuples. Each tuple should contain a unique ordering of column numbers, like (1, 2, 3, 4). Include negative values—for example, (2, -3, 4, -1)—to capture encryption routes that go up columns versus down. A solution is provided in the appendix, with a downloadable version on the book's website in *permutations_practice.py*.

Route Transposition Cipher: Brute-Force Attack

Copy and modify the *route_cipher_decrypt.py* program to hack the route cipher in **Figure 4-2**. Rather than inputting a single key, loop through all possible keys—for an assumed number of columns—and print the results (use the earlier permutations code to generate the keys for this four-column cipher). The impact of switching the order of columns and allowing up-and-down paths through the transposition matrix is clearly illustrated in **Figure 4-10**. The dashed line is the factorial of the number of columns; the solid line captures the effect of reading up columns as well as down (captured by the inclusion of negative values in the key). If you only needed to deal with the factorial of 4, your job as a cryptanalyst would be

easy. But as the cipher gets longer, the number of possible keys explodes. And some actual Union route ciphers had 10 or more columns!

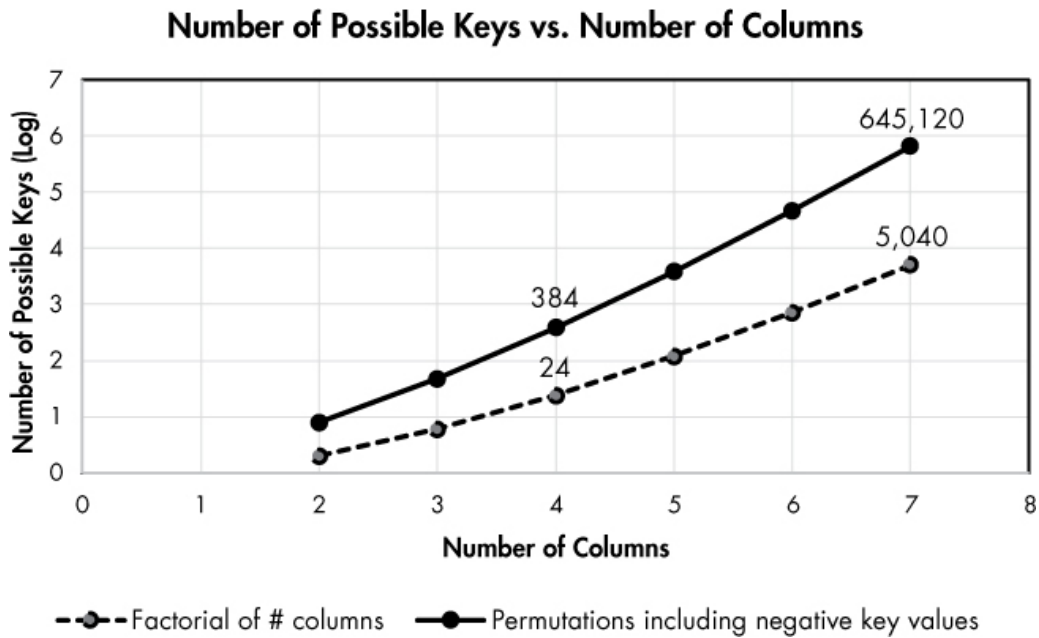


Figure 4-10: Number of possible keys versus number of columns for a route cipher

Here are four translations out of the 384 produced for the ciphertext in

Figure 4-2:

```
using key = [-4, -1, -2, -3]
translated = IS HEADING FILLER VILLAGE YOUR SNOW SOUTH GODWIN ARE FREE TO YOU
WITH SUPPLIES GONE TRANSPORT ROANOKE JUST TO REST

using key = [1, 2, -3, 4]
translated = REST ROANOKE HEADING TO TRANSPORT WITH SNOW GONE YOU ARE FREE TO
GODWIN YOUR SUPPLIES SOUTH VILLAGE IS JUST FILLER

using key = [-1, 2, -3, 4]
translated = VILLAGE ROANOKE HEADING TO GODWIN WITH SNOW GONE YOU ARE FREE TO
TRANSPORT YOUR SUPPLIES SOUTH REST IS JUST FILLER

using key = [4, -1, 2, -3]
translated = IS JUST FILLER REST YOUR SUPPLIES SOUTH TRANSPORT ARE FREE TO YOU
WITH SNOW GONE GODWIN ROANOKE HEADING TO VILLAGE
```

The correct answer is present, but you can appreciate how hard it would be to pick it out quickly, given the use of code words and dummy words. Still, you did your job. Go have that mint julep or some sweet tea.

A solution to this project is provided in the appendix and at <https://www.nostarch.com/impracticalpython/> in `route_cipher_hacker.py`. You will also need the `perms.py` program, which is based on the previous practice project.

Challenge Projects

No solutions are provided for challenge projects.

Route Cipher Encoder

A greenhorn Union telegraph clerk needs to encrypt the following message, complete with code words (**Table 4-1**). Help them out by writing a program that takes the message as input and automatically substitutes the code words, fills the bottom row with dummy words, and transposes the words using the key `[-1, 3, -2, 6, 5, -4]`. Use a 6×7 matrix and make up your own dummy words.

We will run the batteries at Vicksburg the night of April 16 and proceed to Grand Gulf where we will reduce the forts. Be prepared to cross the river on April 25 or 29. Admiral Porter.

Table 4-1: Code Words

Batteries	HOUNDS
Vicksburg	ODOR
April	CLAYTON
16	SWEET
Grand	TREE
Gulf	OWL

Forts	BAILEY
River	HICKORY
25	MULTIPLY
29	ADD
Admiral	HERMES
Porter	LANGFORD

Consider using a Python dictionary for the lexicon of code words in this table.

Three-Rail Fence Cipher

Write a version of the rail fence cipher that uses three rails (rows) instead of two. You can find a hint at

https://en.wikipedia.org/wiki/Rail_fence_cipher.